

MACHINE LEARNING PROJECT REPORT

Fake Job Posting Detection

Using Machine Learning

Identifying Fraudulent Job Advertisements through NLP, Class-Balanced Ensemble Learning, and an Interactive Streamlit Application

0.8529

FINAL F1-SCORE

0.9294

FINAL PR-AUC

17,880

RECORDS ANALYZED

Python · Scikit-learn · XGBoost · TF-IDF · Streamlit · Plotly

SUBMITTED BY

Manish Sahu

Government College of Engineering Kalahandi, Bhawanipatna
ODISHA

Table of Contents

1. Abstract.....	2
2. Introduction.....	2
2.1 Problem Statement	2
2.2 Objectives.....	2
3. Dataset Description.....	2
3.1 Feature Overview	2
4. Methodology	3
4.1 Data Preprocessing.....	3
4.2 Handling Class Imbalance.....	3
4.3 Model Selection	3
5. Model Evaluation and Results	3
5.1 Evaluation Metrics	3
5.2 Results Summary	4
5.3 Discussion of Results	4
6. System Deployment.....	5
6.1 Application Features & UI	5
6.2 Demonstration	5
7. Conclusion and Future Work	8
7.1 Conclusion	8
7.2 Future Enhancements	8
Appendix A: Installation and Usage Guide	9
A.1 Installation.....	9
A.2 Running the Application	9
A.3 Training the Model (Optional)	9
A.4 Dataset Note	9

1. Abstract

The proliferation of fraudulent job postings online poses a significant threat to job seekers, often resulting in financial loss, identity theft, and diminished trust in digital employment platforms. This project presents a machine learning system designed to automatically identify fake job advertisements. By combining unstructured textual data — such as job descriptions and company profiles — with structured categorical features, including employment type and telecommuting status, multiple classification models were trained and evaluated. The final system, powered by an XGBoost classifier, achieved an F1-Score of 0.8529 and was deployed as an interactive Streamlit web application featuring a Cyber Threat Dashboard interface, Plotly-based Radar Threat Matrix visualizations, and enterprise-grade batch CSV analysis for both real-time and bulk fraud screening.

2. Introduction

2.1 Problem Statement

Online job portals are frequently exploited by malicious actors who post fake listings to harvest personal data or execute financial scams. Given the scale of postings on modern platforms, manually reviewing every listing for authenticity is unscalable, creating a clear need for an automated, data-driven detection system.

2.2 Objectives

1. Perform exploratory data analysis on a real-world job posting dataset.
2. Apply Natural Language Processing (NLP) techniques to extract meaning from textual descriptions.
3. Train and compare multiple machine learning algorithms for fraud classification.
4. Deploy the best-performing model as a functional web application for real-time predictions.

3. Dataset Description

The project utilized the Employment Scam Aegean Dataset (EMSCAD), widely known as the “Real / Fake Job Posting Prediction” dataset. The dataset comprises 17,880 job listings and is highly imbalanced, with roughly 95% of postings legitimate and only about 5% fraudulent — a distribution that closely mirrors how rare scams are relative to genuine listings on real-world job platforms.

3.1 Feature Overview

Each listing in the dataset is described by three broad categories of features:

Feature Type	Fields
Textual	<i>title, company_profile, description, requirements, benefits</i>
Categorical	<i>employment_type, required_experience, required_education</i>
Binary	<i>telecommuting, has_company_logo, has_questions</i>

4. Methodology

4.1 Data Preprocessing

- **Missing Value Handling:** Textual columns with missing data were filled with empty strings, while missing categorical values were labeled “Unknown.”
- **Feature Engineering:** All text features were concatenated into a single combined_text column to capture the overall linguistic context of each posting.
- **Vectorization:** Scikit-learn’s TfidfVectorizer (Term Frequency–Inverse Document Frequency) converted the raw text into a numerical matrix, limited to the 5,000 most significant terms.
- **Categorical Encoding:** OneHotEncoder transformed categorical variables into binary vectors.

4.2 Handling Class Imbalance

Because fraudulent postings represent a small minority of the dataset, standard accuracy is a misleading metric — a model that predicts “Legitimate” for every posting would reach 95% accuracy while catching zero scams. To address this, class-weight balancing (class_weight='balanced' and scale_pos_weight) was applied during training, forcing each model to penalize misclassification of the minority “Fraudulent” class more heavily.

4.3 Model Selection

Five candidate algorithms were evaluated:

- **Logistic Regression:** a strong, interpretable baseline model.
- **Multinomial Naive Bayes:** a classic, lightweight standard for text classification.
- **Random Forest:** an ensemble method that handles non-linear relationships well.
- **LinearSVC:** a Support Vector Machine optimized for high-dimensional text data.
- **XGBoost:** a highly optimized gradient boosting framework known for top-tier performance.

5. Model Evaluation and Results

5.1 Evaluation Metrics

Each model was evaluated using Precision, Recall, F1-Score, and Precision–Recall Area Under the Curve (PR-AUC). Recall measures how many of the actual fake jobs were successfully caught, while Precision measures how many of the flagged jobs were genuinely fraudulent — reflecting the rate of false alarms.

5.2 Results Summary

Model	Precision	Recall	F1-Score	PR-AUC
Logistic Regression	0.5532	0.9017	0.6857	0.8817
Naive Bayes	0.8852	0.3121	0.4615	0.6534
Random Forest	0.9903	0.5896	0.7391	0.8930
LinearSVC	0.7906	0.8728	0.8297	0.9215
XGBoost	0.8683	0.8382	0.8529	0.9294

Table 1. Performance comparison across all five candidate models. The XGBoost row (highlighted) was selected for deployment.

5.3 Discussion of Results

- **Naive Bayes** struggled heavily with Recall, catching only about 31% of fake jobs despite reasonable precision.
- **Logistic Regression** achieved excellent Recall ($\approx 90\%$) but very poor Precision ($\approx 55\%$), meaning it flagged far too many legitimate postings as fraudulent.
- **XGBoost** provided the best overall balance, achieving the highest F1-Score (0.8529). It correctly identified roughly 84% of all fake jobs while maintaining a high precision of roughly 87%, making it the model selected for deployment.

6. System Deployment

The final XGBoost model and TF-IDF pipeline were serialized using joblib and integrated into an interactive Streamlit web application, designed and framed as an advanced AI Fraud Detection Engine rather than a simple prediction form.

6.1 Application Features & UI

- **Cyber Threat Dashboard UI:** the interface adopts a high-tech, glassmorphism aesthetic — neon cyan and magenta accents over a dynamic night-sky background — branded as an “AI Fraud Detection Engine” with an “XGBoost Neural Core Active” status badge.
- **Single Analysis & Radar Matrix:** under Payload Parameters, users complete the Content Fields, Metadata Flags, or Quick Presets tabs and run Initiate Threat Scan; the System Diagnosis panel returns a calculated fraud probability alongside an interactive Radar Threat Matrix plotting Linguistic Risk, Missing Metadata, and Suspicious Requirements on a 0–100 scale.
- **Batch Neural Analysis:** a Batch Analysis mode, selectable alongside Single Analysis, lets users upload a CSV file of multiple job postings; the engine scores every listing simultaneously with the XGBoost model and returns both a preview table and a downloadable, comprehensive Threat Report CSV.

6.2 Demonstration

Figures 1 through 3 showcase the redesigned Cyber Threat Dashboard, from its standby state through live analysis of a legitimate and a fraudulent job posting.

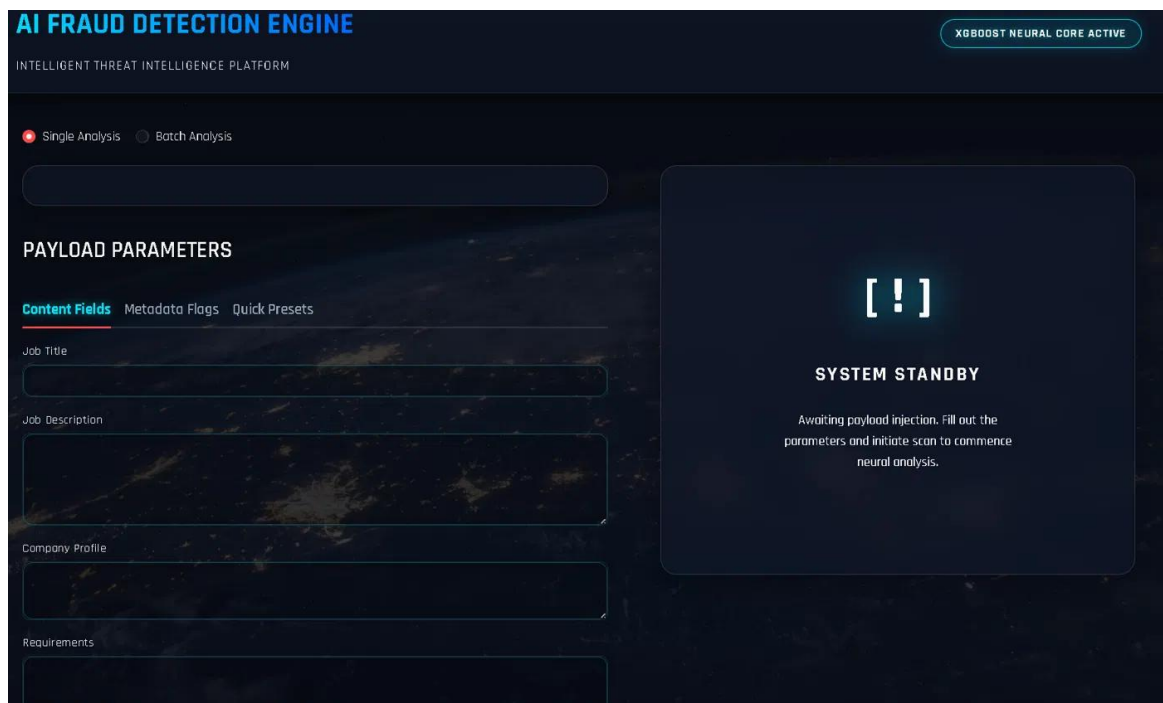


Figure 1. The AI Fraud Detection Engine’s standby interface, awaiting input under the Payload Parameters panel before a scan is initiated.

The dashboard opens in a System Standby state, with the XGBoost Neural Core Active badge confirming the model is loaded and ready. Users choose between Single Analysis and Batch Analysis before completing the Payload Parameters form.

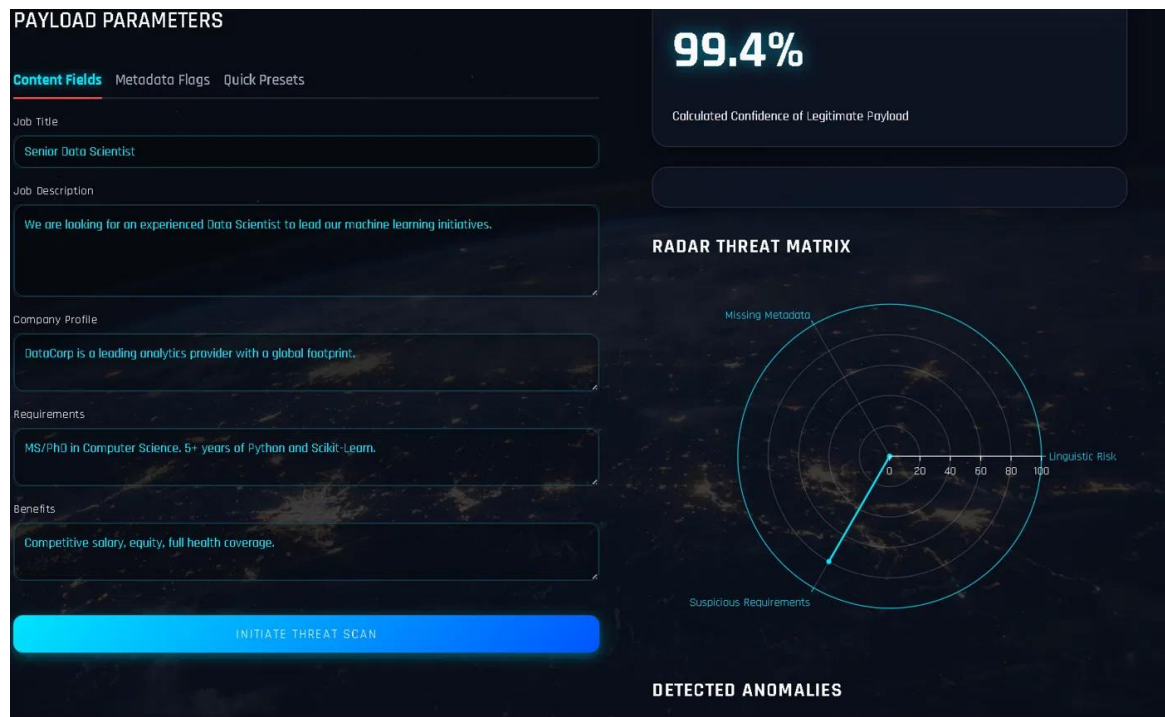


Figure 2. A legitimate “Senior Data Scientist” posting at DataCorp, returning a 99.4% calculated confidence of a legitimate payload with a low, tightly clustered Radar Threat Matrix.

With a complete company profile, realistic qualifications, and standard benefits, the posting scores as legitimate. The Radar Threat Matrix collapses toward the center, reflecting minimal linguistic risk and few suspicious requirements.

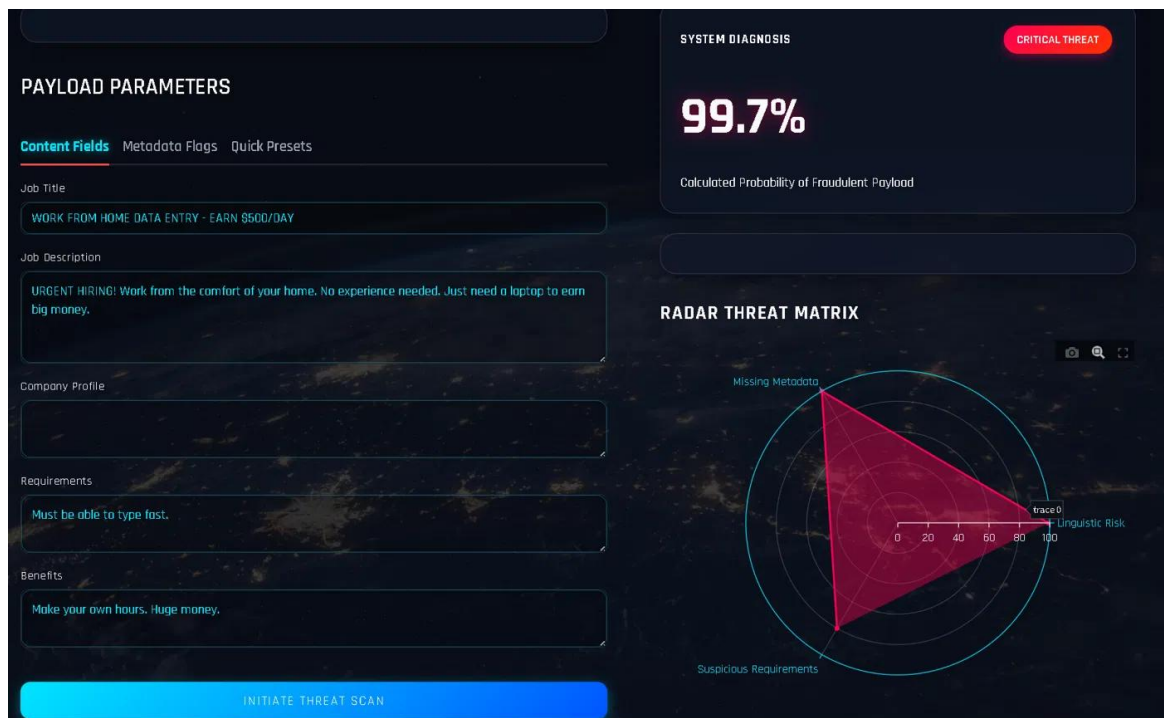


Figure 3. A fraudulent “Work From Home Data Entry” posting, flagged as a Critical Threat with a 99.7% calculated probability of a fraudulent payload.

The listing’s urgent, money-focused language and missing company profile push the Radar Threat Matrix outward across all three axes — Missing Metadata, Linguistic Risk, and Suspicious Requirements — visually confirming why the system escalated it to Critical Threat.

7. Conclusion and Future Work

7.1 Conclusion

This project demonstrates that machine learning and NLP techniques can effectively separate fraudulent job postings from legitimate ones. By combining TF-IDF text vectorization with an XGBoost classifier trained under balanced class weights, the system achieved a high degree of reliability, confirming that linguistic patterns and missing metadata — such as absent company logos or profiles — are strong indicators of fraud. Packaging the model as a Cyber Threat Dashboard with Radar Matrix visualization and batch CSV processing further demonstrates that the underlying approach can scale from single-listing checks to enterprise-grade, bulk screening workflows.

7.2 Future Enhancements

- **Deep Learning / LLMs:** replacing TF-IDF with advanced embeddings such as BERT, or fine-tuning a small large language model, could capture deeper semantic meaning in scam descriptions.
- **Explainable AI (XAI):** integrating SHAP (SHapley Additive exPlanations) into the Streamlit app to highlight the exact words (e.g., “urgent,” “wire transfer”) that caused a specific posting to be flagged.
- **Web Scraping:** extending the system to accept a URL from LinkedIn or Indeed, automatically scraping the listing text and returning a risk score without manual entry.

Appendix A: Installation and Usage Guide

A.1 Installation

1. Clone the repository:

```
git clone <your-repo-url>
cd Fake-Job-Posting-Detection
```

2. Create a virtual environment (optional but recommended):

```
python -m venv venv
source venv/bin/activate # On Windows use: venv\Scripts\activate
```

3. Install the dependencies:

```
pip install -r requirements.txt
```

A.2 Running the Application

To start the Streamlit web application, run the following command from the project root:

```
streamlit run app.py
```

This launches the application in the default web browser, typically at <http://localhost:8501>.

A.3 Training the Model (Optional)

To retrain the model or test additional classifiers (Logistic Regression, Random Forest, etc.), run the training script. It automatically downloads the dataset if missing, trains all models, and outputs evaluation metrics.

```
python train.py
```

A.4 Dataset Note

The project uses the “Real / Fake Job Posting Prediction” dataset (EMSCAD). The training script handles downloading this dataset automatically from a raw open-source mirror.